

Динамическая персонализация без состояния: от псевдослучайных чисел до алгоритмической адаптации **LLM**

Фундаментальный Принцип: Детерминированная Генерация Псевдослучайных Последовательностей

В основе решения задачи по динамической генерации уникальных последовательностей цифр, адаптируемых под конкретного клиента и не зависящих от внешнего мира, лежит фундаментальный принцип детерминированных генераторов псевдослучайных чисел (ГПСЧ). Этот подход представляет собой наиболее прямолинейное и эффективное решение, полностью соответствующее всем трем ключевым требованиям исследовательской цели: персонализация, немедленная адаптация и полное игнорирование реального мира. ГПСЧ — это детерминированные алгоритмы, которые производят последовательности чисел, которые кажутся случайными, но на самом деле полностью предсказуемы, если известны начальные условия [12](#). Это свойство делает их идеальным инструментом для создания систем, где необходима одновременная индивидуализация и абсолютная воспроизводимость, что является критически важным требованием для многих корпоративных и научных приложений.

Центральная характеристика любого ГПСЧ заключается в его детерминизме: для одного и того же входа (начального семени) алгоритм всегда будет генерировать одинаковую последовательность выходных значений [7](#) [11](#). Это определяет два ключевых элемента системы: начальное значение и сам алгоритм. Начальное значение, или «семя», является своего рода "ключом", который "перемешивает" фиксированный набор чисел, порожденный алгоритмом, и запускает процесс генерации с определенной точки [12](#). Поскольку вся последующая последовательность является математической функцией от предыдущего значения, выбор семени напрямую определяет все будущие результаты. Это свойство позволяет легко персонализировать генерацию для каждого клиента. Вместо того чтобы иметь единую, обобщенную последовательность, система может использовать уникальное семя для

каждого конкретного клиента. Различные семена будут порождать совершенно разные, непересекающиеся последовательности псевдослучайных чисел, обеспечивая каждому клиенту собственный, уникальный поток данных ¹⁷. Например, в C++ библиотеке `stateless_rand` для этой цели используется статический метод `seed(seed_value)`, который инициализирует новый экземпляр генератора, уже находящийся в состоянии, соответствующем этому семени ¹⁷.

Механизм немедленной адаптации в рамках этого подхода чрезвычайно прост и быстр. Когда система получает запрос от нового клиента или когда у существующего клиента меняются параметры, определяющие его идентичность, она просто обращается к своей базе данных или конфигурации, чтобы получить уникальное семя, назначенное этому клиенту. После получения семени генератор начинает свою работу, производя первую цифру последовательности. Этот процесс занимает микроскопическое количество времени, так как он сводится к одной математической операции — применению формулы генератора к текущему состоянию. Нет необходимости в обучении, сложных вычислениях или доступе к внешним данным. Адаптация происходит мгновенно, поскольку весь необходимый контекст (семя) передается вместе с запросом или извлекается из легкодоступного источника. Более того, некоторые реализации ГПСЧ, такие как `stateless_rand`, специально спроектированы для функционального программирования, где состояние не изменяется "на месте". Вместо этого вызов метода `next()` возвращает новое, обновленное состояние генератора, представляющее следующее число в последовательности ¹⁷. Такой подход устраняет проблемы, связанные с совместным использованием общего изменяемого состояния в многопоточных средах, и упрощает управление жизненным циклом генераторов, что особенно важно в распределенных системах.

Условие "без учета реального мира" выполняется идеально.

Последовательность, генерируемая ГПСЧ, является замкнутой системой, полностью определяемой ее внутренним состоянием и начальным семенем. Она не зависит от времени суток, погодных условий, рыночных колебаний или любых других внешних факторов. Единственным источником "случайности" является само начальное семя, которое должно быть выбрано таким образом, чтобы обеспечить равномерное распределение и длинный период повторения, но само по себе оно не является случайным в смысле внешнего события. Для обеспечения высокого качества псевдослучайности рекомендуется использовать хорошо изученные и протестированные алгоритмы, такие как

линейный конгруэнтный генератор (ЛКГ), который используется в стандартной библиотеке C++ (`std::minstd_rand`) [17](#), или более сложные, но статистически более надежные методы, например, на основе хеш-функций SHA-256 [19](#). Использование `/dev/random` или `/dev/urandom` на Linux-системах также является распространенной практикой для получения истинно случайного источника энтропии для инициализации единственного "главного" ГПСЧ в приложении, но сама генерация последовательностей должна происходить исключительно на основе этого семени [12](#). Это гарантирует, что даже при использовании внешней энтропии для первоначальной настройки, дальнейшая работа системы остается полностью автономной и не зависит от внешних событий.

Рассмотрим архитектуру на основе ГПСЧ. В простейшем случае это может быть состоятельная служба, которая для каждого клиента хранит свое состояние ГПСЧ. Однако гораздо более масштабируемым и отказоустойчивым решением является состоятельная архитектура. В этом сценарии каждый запрос от клиента должен содержать всю необходимую информацию для генерации. Наиболее распространенный способ — это передача идентификатора клиента или самого уникального семени в теле запроса (например, в заголовке или JSON-объекте). Сервер, получив такой запрос, использует идентификатор клиента для быстрого поиска его семени в базе данных или кеше. После этого он применяет выбранный алгоритм ГПСЧ к этому семени, чтобы сгенерировать одну или несколько цифр. Результат отправляется клиенту, а сервер очищает все промежуточные данные. Следующий запрос от того же клиента, содержащий тот же идентификатор, будет обработан аналогично, но для продолжения последовательности потребуется сохранить текущее состояние генератора между вызовами. Поскольку состояние генератора обычно представляет собой одно число, его можно легко сохранять в том же внешнем хранилище (например, Redis), связывая его с идентификатором клиента. Таким образом, при следующем вызове система сможет быстро прочитать предыдущее состояние, продолжить генерацию и сразу же записать новое состояние обратно в хранилище. Этот гибридный подход сочетает масштабируемость состоятельных серверов с возможностью персонализации, основанной на внешнем управляемом состоянии [9](#) [15](#).

Другой интересный вариант реализации — использование хеш-функций для генерации. Например, можно взять начальный seed и номер образца (например, 1, 2, 3...), объединить их в строку (например, "my_seed,1"), вычислить SHA-256 хеш этой строки и интерпретировать полученный шестнадцатеричный

результат как большое число ¹⁹. Остаток от деления этого числа на максимальное желаемое значение даст нам случайное число в нужном диапазоне. Преимущество такого подхода в том, что он позволяет "перескакивать" по последовательности. Если мы хотим десятое число в последовательности клиента с определенным семенем, нам не нужно генерировать первые девять чисел; достаточно вычислить хеш для "seed,10" и получить результат напрямую. Это свойство называется "прямым доступом" и может быть полезно в некоторых сценариях. Кроме того, качество такого генератора напрямую связано с качеством используемой хеш-функции, которая, как известно, имеет хорошие статистические свойства равномерного распределения ¹⁹.

Несмотря на свои очевидные преимущества, подход на основе ГПСЧ имеет и ограничения. Главный из них — это качество "случайности" и длина периода. Хотя многие современные ГПСЧ проходят строгие статистические тесты, они не являются истинно случайными и их последовательности рано или поздно повторяются ¹². Выбор алгоритма имеет решающее значение. Простые ГПСЧ, такие как линейные конгруэнтные генераторы, имеют относительно короткие периоды и могут демонстрировать некоторые закономерности в многомерном пространстве, что делает их непригодными для задач, требующих высокой степени истинной случайности, например, в криптографии. Для таких задач требуются специализированные алгоритмы, такие как ГПСЧ Мерсенна-Пирамиды. Однако для большинства задач персонализации, симуляций и генерации тестовых данных, где важна не криптостойкость, а лишь кажущаяся случайность и воспроизводимость, хорошо спроектированные ГПСЧ являются более чем достаточным инструментом. Еще одним практическим аспектом является управление семенами. Необходима надежная система для генерации, хранения и управления уникальными семенами для каждого клиента, чтобы гарантировать отсутствие коллизий и сохранение целостности персонализации. Любая ошибка в этой системе может привести к тому, что разные клиенты получат одинаковые последовательности или одна и та же последовательность будет разрываться для одного клиента.

В итоге, детерминированная генерация псевдослучайных чисел представляет собой мощную, эффективную и теоретически чистую модель для достижения поставленной исследовательской цели. Она демонстрирует, как можно достичь динамической, персонализированной и адаптивной генерации в замкнутой, состоятельной среде. Скорость адаптации достигается за счет простоты математической операции, а зависимость от реального мира исключается

путем полного контроля над источником случайности (семенем). Эта парадигма служит фундаментом, понимание которого необходимо для осмысления более сложных и современных подходов, таких как адаптация больших языковых моделей через приписки или генерация детерминированного кода.

Алгоритмическая Адаптация Лингвистических Моделей: Роль Приписок

Хотя генераторы псевдослучайных чисел (ГПСЧ) предлагают элегантное и эффективное решение для персонализации числовых последовательностей, современный ландшафт искусственного интеллекта открывает новые возможности для адаптации поведения сложных систем, таких как большие языковые модели (БЯМ). В частности, методология, основанная на добавлении к входному запросу специальных последовательностей токенов — так называемых "приписок" (суффиксов), — представляет собой мощный инструмент для немедленной и точной адаптации модели под уникальные требования клиента. Этот подход, детально описанный в исследованиях по Adaptive Content Restriction (AdaCoRe) и атакам типа "джейлбрейк", полностью соответствует целям данного исследования: он обеспечивает персонализацию, работает мгновенно и не требует модификации самой модели или доступа к внешнему миру во время выполнения [2](#) [3](#) .

Суть метода заключается в том, что вместо того, чтобы пытаться изменить веса или архитектуру уже обученной LLM, что является дорогостоящей и нетривиальной задачей, система работает с моделью "извне". Она принимает на вход исходный пользовательский запрос и добавляет к нему короткую, заранее подготовленную последовательность токенов — суффикс. Этот суффикс не является частью оригинального запроса пользователя; он является специальным "контрольным сигналом", который был получен в результате сложного алгоритмического процесса оптимизации [2](#) . Цель этого процесса — найти такую последовательность токенов, которая, будучи добавленной к любому запросу, будет искусственно направлять или "контролировать" вывод модели в желаемом направлении. Например, в задаче AdaCoRe целью является поиск суффикса, который минимизирует вероятность генерации определенных "запрещенных" терминов, сохраняя при этом высокое качество и семантическую релевантность ответа [2](#) . В контексте атак на безопасность (джейлбрейк) цель,

наоборот, состоит в том, чтобы найти суффикс, который максимизирует вероятность генерации опасного или нежелательного контента ³.

Процесс создания такого "оптимизированного суффикса" является офлайн-операцией, то есть он выполняется один раз, до начала работы системы в реальном времени. Он представляет собой сложную задачу дискретной оптимизации, где искомой переменной является сам суффикс (последовательность токенов) ³. Для решения этой задачи используется специализированный алгоритм, часто являющийся вариацией градиентного спуска, но адаптированный для работы с дискретными объектами (токенами). Наиболее распространенным методом является Greedy Coordinate Gradient (GCG) и его вариации ³. Алгоритм работает итерационно: на каждой итерации он рассматривает одну позицию в суффиксе и пробует заменить текущий токен на другие возможные токены из словаря модели, выбирая ту замену, которая дает наибольший прогресс в достижении целевой функции (например, максимальное снижение вероятности генерации запрещенного слова). Этот процесс продолжается до тех пор, пока не будет найдена оптимальная или достаточно хорошая последовательность. Целевая функция (loss function), которую оптимизирует алгоритм, обычно является комбинацией нескольких компонентов. В случае SOP, это трехкомпонентная функция, включающая: 1) loss ограничения, направленный на подавление запрещенных терминов; 2) quality loss, поддерживающий высокое качество ответа; и 3) semantic loss, обеспечивающий сохранение контекстной релевантности ². Этот многоцелевой подход позволяет добиться баланса между жестким контролем и естественностью ответа.

Ключевое преимущество этого подхода в контексте исследования заключается в его способности обеспечивать "немедленную адаптацию". Процесс оптимизации суффикса, хотя и вычислительно затратен (например, на одном из экспериментов для модели LLaMA-3.1-8B требовалось около 16.88 минут и 42 ГБ памяти GPU A100 ²), является однократным. После того как суффикс создан и сохранен, его применение к любому запросу клиента происходит практически мгновенно. Сама по себе операция "конкатенация строки" является одной из самых быстрых операций в компьютерных науках. Таким образом, система может иметь набор предварительно оптимизированных суффиксов, каждый из которых предназначен для определенной группы клиентов или для выполнения определенной задачи. При получении запроса от клиента система просто выбирает соответствующий ему суффикс и добавляет его к запросу перед

отправкой в LLM. Адаптация происходит без задержек, характерных для обучения или сложных вычислений в реальном времени.

Персонализация клиента достигается путем сопоставления каждого клиента (или его сегмента) с конкретным, уникальным суффиксом. Например, клиенту А может быть назначен суффикс, который заставляет модель говорить на формальном юридическом языке, клиенту В — суффикс, который активирует "творческий режим", а клиенту С — суффикс, который блокирует определенные категории информации. Этот механизм очень гибок и позволяет реализовать сложные, многоуровневые политики адаптации. Интересной особенностью является и то, что суффиксы, оптимизированные для одной модели, могут демонстрировать высокую "переносимость" (transferability) на другие модели ². В одном из примеров суффикс, оптимизированный на модели Llama3, показал заметную эффективность при подавлении атак на модели Mistral и Vicuna ². Это говорит о том, что существует некий универсальный механизм воздействия, который можно эксплуатировать, создавая даже "универсальные" адаптации, которые будут работать на широком спектре LLM.

Механизм действия суффикса лежит в плоскости архитектуры трансформеров. Когда суффикс добавляется к входному запросу, он становится частью входного контекста модели. Исследования показывают, что эти искусственно добавленные токены оказывают доминирующее влияние на внимания на последних слоях трансформера, фактически "перехватывая" процесс предсказания следующего токена ³. Они заставляют модель "забыть" свои изначальные настройки и следовать новому, искусственно созданному пути генерации. Это позволяет имитировать изменения в поведении модели без каких-либо изменений в ее параметрах. Для защиты от таких атак могут использоваться "защитные суффиксы", которые оптимизируются для минимизации вероятности генерации вредоносного контента. Их применение продемонстрировало способность снижать успешность атак до 79% с минимальным влиянием на обычное использование модели ³.

Этот подход идеально соответствует ограничению "без учета реального мира". Адаптация происходит исключительно на основе структуры и содержания суффикса. Система не собирает никаких данных о взаимодействиях с клиентом для обучения или адаптации. Все знания, необходимые для адаптации, закодированы в самом суффиксе. Это делает процесс полностью детерминированным и воспроизводимым. Для одного и того же запроса и

одного и того же суффикса LLM всегда будет генерировать один и тот же ответ, что является важным свойством для аудита и отладки.

Сравнивая этот метод с ГПСЧ, можно выделить несколько ключевых различий. ГПСЧ генерирует числовые последовательности, в то время как суффиксы адаптируют поведение сложной языковой модели. ГПСЧ требует простого числового семени, тогда как суффикс — это короткая текстовая последовательность. ГПСЧ является простой математической функцией, тогда как эффект суффикса является результатом сложной оптимизации и глубоко укоренился в нелинейной динамике трансформера. Однако оба подхода разделяют общую концепцию: динамическое, персонализированное поведение достигается за счет предоставления на входе дополнительного "контрольного сигнала", который полностью определяет дальнейшую траекторию системы. В случае ГПСЧ это семя, в случае суффиксов — оптимизированная последовательность токенов.

В практическом плане реализации система, использующая суффиксы, должна включать два основных компонента: офлайн-процесс оптимизации и онлайн-сервис применения. Оптимизация может быть запущена разово или периодически для создания новых адаптаций. Результаты (сгенерированные суффиксы) сохраняются в базе данных. Онлайн-сервис, получая запрос, использует бизнес-логику для определения клиента, выбирает соответствующий ему суффикс из базы и выполняет простую операцию конкатенации перед отправкой запроса в LLM. Эта архитектура относительно проста в реализации и не требует значительных изменений в существующей инфраструктуре LLM.

Таким образом, адаптация через приписки представляет собой современную, мощную и эффективную альтернативу классическим ГПСЧ. Она позволяет осуществлять мгновенную, персонализированную и чисто алгоритмическую адаптацию сложных ИИ-систем, полностью укладываясь в рамки замкнутой системы без связи с реальным миром. Это демонстрирует переход от простой генерации чисел к более сложной и гибкой форме управления поведением искусственного интеллекта.

Парадигма "Скомпилированного ИИ": От Генерации к Детерминированному Коду

В рамках исследования динамической генерации и адаптации в замкнутых системах, парадигма "Скомпилированного ИИ" (Compiled AI) представляет собой радикальный и высокоэффективный подход, который кардинально меняет саму метафору взаимодействия с большой языковой моделью (БЯМ). В отличие от традиционных методов, где LLM вызывается для каждого запроса в реальном времени (интерпретация), Compiled AI предлагает модель, в которой LLM используется только один раз — в момент "компиляции" — для создания статического, исполняемого кода. После этого развертывания рабочий процесс выполняется детерминированно, без каких-либо дальнейших обращений к LLM [10](#). Этот метод полностью устраняет состояние во время выполнения, обеспечивает максимальную скорость, предсказуемость и экономию ресурсов, что делает его идеальным кандидатом для анализа в контексте поставленной исследовательской задачи.

Основная идея Compiled AI заключается в том, чтобы извлечь из LLM бизнес-логику и преобразовать ее в машинно-читаемый, оптимизированный код. Процесс начинается с того, что человек-разработчик описывает желаемую задачу с помощью простого декларативного языка, например, YAML-файла, в котором указываются входные данные, целевые действия и конечная цель [10](#). Этот YAML-файл вместе с описанием доступных "строительных блоков" (компонентов из библиотеки шаблонов и модулей) формируется в комплексное промпт-сообщение. Затем этот промпт отправляется в LLM, которая генерирует не текстовый ответ, а готовый к исполнению программный код (например, Python-скрипт) [10](#). Этот код уже содержит всю необходимую логику: обработку данных, вызовы API, работу с базами данных, условные переходы. После генерации этот код проходит через строгий четырехэтапный пайплайн проверки перед развертыванием [10](#).

Четырехэтапный пайплайн проверки является ключевым элементом, обеспечивающим безопасность и надежность системы: 1. Безопасность: Код подвергается статическому анализу на предмет уязвимостей, таких как SQL-инъекции или командная инъекция [10](#). 2. Синтаксис: Проверяется синтаксическая корректность, типизация и соответствие стандартам кодирования (linting) [10](#). 3. Выполнение: Код выполняется в изолированной среде (песочнице) на наборе тестовых данных (фикстурах) для проверки его

работоспособности [10](#). 4. Точность: Результат выполнения сравнивается с "золотыми" эталонными данными с использованием задачеспецифичных порогов точности [10](#).

Только после успешного прохождения всех этих этапов код считается валидным и может быть развернут. Важно отметить, что после развертывания LLM больше не участвует в процессе. Рабочий процесс выполняется как обычная программа, что обеспечивает полную детерминированность и отсутствие вариативности, присущей runtime-выводам LLM [10](#).

Этот подход напрямую решает проблему "динамической генерации" и "персонализации под клиента". Персонализация достигается на этапе "компиляции". Для каждого клиента или для каждой уникальной бизнес-логики система может запустить процесс генерации кода заново с другими параметрами или шаблонами. В результате для разных клиентов могут быть сгенерированы совершенно разные, но одинаково надежные и быстрые рабочие процессы. Это пример "динамической генерации", которая происходит не в реальном времени, а в момент подготовки системы к работе. "Немедленная адаптация" здесь трактуется как возможность мгновенного переключения на другой, уже скомпилированный и проверенный рабочий процесс, который был подготовлен заранее. Это не адаптация в реальном времени, а выбор из набора готовых, оптимизированных решений.

Преимущества Compiled AI в контексте исследования очевидны. Во-первых, это скорость. Поскольку во время выполнения не требуется обращений к LLM, латентность снижается с сотен миллисекунд или даже секунд до долей миллисекунды. В одном из тестов P50 латентность Compiled AI составила всего 4.5 мс по сравнению с 2004 мс у прямого вызова LLM [10](#). Во-вторых, это предсказуемость. Runtime-выводы LLM, даже от одной и той же модели, могут давать разные результаты из-за стохастической природы процесса декодирования (up to 15% accuracy variation across runs was observed in one experiment [10](#)). Compiled AI, выполняя статический код, обеспечивает 100% репродуктивность и нулевую вариативность вывода (zero output entropy) [10](#). Это критически важно для финансовых расчетов, обработки документов и других задач, где требуется высокая точность. В-третьих, это экономическая эффективность. Из-за полного отсутствия runtime-вызовов к LLM стоимость владения системой на уровне миллионов транзакций в месяц может быть в десятки раз ниже, чем у систем, работающих в реальном времени [10](#).

Связь с ограничением "без учета реального мира" здесь еще более выражена. Сгенерированный код — это замкнутая система, которая выполняет свои действия на основе жестко заложенной в него логики и данных, предоставленных ей во время выполнения. Она не "учится" и не "адаптируется" в процессе работы, как это делают агентские системы. Это делает ее абсолютно стабильной и неподверженной внешним влияниям. Однако эта парадигма не является универсальной. Она лучше всего подходит для задач, где бизнес-логика четко определена и не требует сложных runtime-решений. Для задач, требующих некоторой степени гибкости, система поддерживает "ограниченное агентское использование" (bounded agentic invocation), где сгенерированный детерминированный код может вызывать LLM для выполнения специфических, ограниченных подзадач (например, извлечения структурированных данных из неструктурированного текста), но при этом сама общая логика остается детерминированной с четко определенными fallback-вариантами и механизмами мониторинга ¹⁰.

Сравнивая Compiled AI с двумя предыдущими подходами, можно выстроить следующую иерархию сложности и гибкости:

Характеристика	Генератор псевдослучайных чисел (ГПСЧ)	Алгоритмическая адаптация (суффиксы)	Скомпилированный ИИ
Объект адаптации	Числовая последовательность	Поведение большой языковой модели (LLM)	Полный рабочий процесс (код)
Механизм персонализации	Уникальное семя для каждого клиента ¹⁷	Уникальная оптимизированная приписка для каждого клиента ²	Генерация уникального детерминированного кода для каждого клиента/задачи ¹⁰
Скорость адаптации	Мгновенная (математическая операция)	Мгновенная (конкатенация строки)	Мгновенная (переключение на другой скомпилированный процесс)
Вычислительная сложность	Очень низкая	Низкая (однократная высокая офлайн-оптимизация) ²	Очень высокая (однократная сложная генерация + строгая проверка) ¹⁰
Гибкость и сложность	Низкая (ограничена алгоритмом ГПСЧ)	Средняя (зависит от длины и сложности суффикса)	Высокая (может генерировать сложные рабочие процессы)
Предсказуемость	Высокая (детерминированная)	Средняя (детерминированная для заданного суффикса, но LLM стохастична)	Очень высокая (полностью детерминированная)

Таким образом, "Скомпилированный ИИ" является экстремальным, но чрезвычайно важным примером, иллюстрирующим переход от динамической генерации "на лету" к детерминированному исполнению. Он демонстрирует, как

можно достичь максимальной производительности, предсказуемости и экономической эффективности, жертвуя гибкостью, присущей агентским системам. Этот подход идеально подходит для задач, где требования к стабильности и скорости перевешивают потребность в runtime-адаптации. В контексте данного исследования он расширяет понимание того, как можно реализовать "динамическую генерацию" в замкнутой системе, показывая, что это может быть не только генерация последовательностей, но и генерация целых, оптимизированных и безопасных программных систем.

Архитектурные Основы: Состоятельность против Состоятельности в Системах Персонализации

Выбор архитектурной парадигмы — состоятельной или состоятельности — является фундаментальным решением, которое определяет масштабируемость, отказоустойчивость, сложность и возможности персонализации любой программной системы. В контексте задачи динамической генерации персонализированных данных для каждого клиента, понимание различий и компромиссов между этими двумя подходами критически важно. Хотя исследовательская цель предполагает фокус на внутренней логике и алгоритмах, архитектура определяет, как эта логика будет организована и как система будет управлять информацией о клиентах для обеспечения персонализации.

Состоятельная архитектура — это модель, в которой сервер хранит и отслеживает состояние (контекст) каждого клиента между запросами [8](#) [13](#). Это означает, что после первого запроса сервер сохраняет информацию о клиенте, например, его идентификатор сессии, предпочтения или прогресс в какой-либо задаче. При последующих запросах от того же клиента сервер обращается к своему внутреннему хранилищу, чтобы восстановить этот контекст и продолжить работу, создавая ощущение непрерывного диалога или взаимодействия [9](#). Типичные примеры состоятельных систем — это традиционные веб-приложения с системами входа в сеть (где сервер помнит, что пользователь авторизован), корзины покупок в электронной коммерции (которые "запоминают" добавленные товары даже при переходе между страницами) и базы данных [9](#) [13](#). В контексте нашей задачи, состоятельная система могла бы хранить для каждого клиента текущее состояние генератора

псевдослучайных чисел (ГПСЧ), чтобы каждый новый вызов возвращал следующее число в его последовательности. Это обеспечивает простоту реализации внутри сервиса, но создает серьезные проблемы с масштабированием и отказоустойчивостью. Поскольку состояние привязано к конкретному серверу, для корректной работы системы требуется механизм "липких сессий" (sticky sessions), который направляет все запросы от одного и того же клиента на один и тот же сервер. Это усложняет архитектуру балансировщика нагрузки и создает узкие места: если сервер с активной сессией клиента выходит из строя, вся информация о состоянии для этого клиента теряется, и ему придется начинать сессию заново [13](#) [15](#).

Состоятельная архитектура, напротив, рассматривает каждый запрос от клиента как независимую транзакцию [8](#) [15](#). Сервер не хранит никакой информации о предыдущих взаимодействиях с этим клиентом. Чтобы обработать запрос, сервер должен получить всю необходимую ему информацию либо из самого запроса (например, в заголовках или теле), либо обратиться к внешним источникам данных, таким как базы данных или кеш [13](#). RESTful API, сетевые службы доставки контента (CDN) и микросервисы часто используют состоятельную архитектуру [13](#). В нашем случае это означает, что для генерации персонализированной последовательности каждый запрос должен нести в себе всю необходимую информацию. Например, запрос должен содержать уникальный идентификатор клиента. Сервер по этому идентификатору обращается к внешнему хранилищу (например, базе данных или Redis), чтобы получить или обновить состояние генератора для этого клиента, затем выполняет генерацию и обновляет состояние обратно. Основное преимущество состоятельной архитектуры — это высочайшая масштабируемость и отказоустойчивость. Поскольку ни один запрос не зависит от состояния другого или от конкретного сервера, можно легко добавлять или удалять серверные экземпляры для обработки пиковой нагрузки. Любой сервер в пуле может обработать любой запрос, что упрощает балансировку нагрузки и повышает общую надежность системы: сбой одного сервера не приведет к потере состояния клиентов [9](#) [15](#).

Однако чисто состоятельные системы плохо подходят для задач, требующих сложной персонализации и контекстной памяти. Именно поэтому большинство современных сложных приложений используют гибридную модель [13](#) [15](#). В этой модели сами приложение-серверы остаются состоятельными для обеспечения масштабируемости, но управление состоянием выносится за их пределы в специализированные, централизованные и часто состоятельные хранилища [9](#).

К таким хранилищам относятся распределенные кеш-системы (например, Redis, Memcached) или внешние базы данных ⁹. Это позволяет системе сочетать лучшее из двух миров: простоту и масштабируемость состоятельных серверов с возможностью поддерживать сложное, персонализированное состояние клиента.

В рамках этой гибридной архитектуры можно реализовать все рассмотренные ранее методы персонализации:

1. ГПСЧ: Каждый клиент имеет свой уникальный идентификатор. Также у каждого клиента есть уникальное "семя" для ГПСЧ. Состояние генератора (текущее значение в последовательности) хранится в Redis, используя идентификатор клиента в качестве ключа. При каждом запросе состоятельный сервис читает текущее состояние из Redis, вычисляет следующее число, записывает новое состояние обратно в Redis и отправляет результат. Этот подход обеспечивает как масштабируемость, так и персонализацию.
2. Адаптация через суффиксы: Здесь персонализация хранится в виде сопоставления идентификатора клиента и его "оптимизированного суффикса". Это сопоставление также можно хранить в Redis или базе данных. Когда сервис получает запрос, он извлекает идентификатор клиента, получает из хранилища его суффикс, добавляет его к запросу и отправляет в LLM. Состояние здесь минимально, так как суффикс является постоянным для клиента.
3. "Скомпилированный ИИ": В этом случае система может хранить в базе данных идентификатор клиента и ссылку на его "скомпилированный" рабочий процесс (код). При получении запроса сервис выбирает правильный рабочий процесс и запускает его. Здесь состояние клиента может храниться отдельно в Redis или базе данных и передаваться в исполняемый код.

Архитектура также определяет, как система может симулировать "состоятельность" и "dynamism", даже будучи состоятельной. Например, в агентских системах, которые стремятся к состоятельному поведению, для управления несколькими уровнями памяти используются общие объекты состояния, которые передаются из одного узла (шага) в другой в рамках графа выполнения (например, DAG) ¹⁴. В нашем случае, состоятельная система может

передавать весь необходимый контекст с каждым запросом, имитируя состояние. Например, запрос может содержать не только идентификатор клиента, но и последний сгенерированный номер, что позволит любому серверу продолжить последовательность. Однако это менее эффективно, чем использование централизованного хранилища, так как увеличивает размер запросов и нагрузку на сеть.

Важно также понимать, что даже внутри LLM, которые считаются состоятельными и состоятельными, возникают попытки создания состоятельности на прикладном уровне [5](#). Приложение, окружающее LLM, может мониторить его вывод на наличие специальных инструкций (вызовов функций), делать паузу, выполнять действие (например, обращаться к базе данных или API), получать результат и передавать его обратно в LLM для продолжения ответа [5](#). Это создает иллюзию непрерывного, состоятельного и целенаправленного агента, в то время как сама LLM остается состоятельной машиной для генерации текста [5](#) [14](#). Этот подход также можно использовать для нашей задачи: состоятельная система может быть просто фасадом над состоятельной LLM, который управляет "состоянием" генерации, передавая его в качестве дополнительного контекста в каждый вызов.

Таким образом, архитектурный выбор между состоятельной и состоятельной моделями является ключевым компромиссом между простотой управления состоянием и масштабируемостью. Для задачи динамической персонализации в замкнутой системе гибридная модель является наиболее практичным и надежным решением. Она позволяет использовать состоятельные, масштабируемые серверы для выполнения основной логики, вынося управление состоянием клиента в надежные внешние хранилища. Это обеспечивает возможность реализации сложных персонализированных сценариев, от генерации псевдослучайных последовательностей до адаптации поведения LLM через суффиксы, при этом сохраняя высокую производительность и отказоустойчивость всей системы.

Синтез и Сравнительный Анализ Методологий

Подводя итог проведенному исследованию, можно сделать вывод, что задача динамической генерации персонализированных цифровых результатов, адаптируемых немедленно и не учитывающих реальный мир, решается через

создание замкнутых, алгоритмически управляемых систем. Ключевая концепция, объединяющая все рассмотренные подходы, заключается в том, что "динамика" и "персонализация" достигаются не за счет хранения долгосрочного состояния или использования внешних данных, а через внедрение в систему специальных "контрольных сигналов" на входе. Эти сигналы, будь то семя для генератора псевдослучайных чисел, оптимизированная приписка для языковой модели или целый блок кода, генерируемый ИИ, полностью определяют поведение системы для каждого конкретного случая. "Немедленная адаптация" в данном контексте означает не адаптацию в реальном времени, а способность мгновенно применять заранее подготовленную и оптимизированную адаптацию, что делает процесс практически мгновенным. Исследование четко сфокусировано на внутренней логике и алгоритмах, что исключает из рассмотрения системы, основанные на сборе и анализе эмпирических данных из реального мира.

На основе проведенного анализа можно выделить три основные методологии, различающиеся по уровню сложности, гибкости и вычислительным затратам. Эти методы можно представить как спектр, от простого и эффективного до сложного и чрезвычайно производительного.

1. Детерминированные Генераторы Псевдослучайных Чисел (ГПСЧ): Простой и Эффективный Основной Уровень. Этот подход является наиболее фундаментальным и прямолинейным. Он основан на детерминированных математических алгоритмах, которые по уникальному "семени" порождают уникальную и воспроизводимую последовательность чисел [12](#) [17](#).

Персонализация достигается простым сопоставлением каждого клиента с уникальным семенем [17](#). Преимущества этого метода очевидны: чрезвычайно высокая скорость генерации (одна математическая операция), полная предсказуемость и воспроизводимость, а также минимальные вычислительные затраты. Этот подход идеально подходит для задач, где требуется генерация числовых последовательностей для тестирования, симуляций, аннулирования или создания уникальных идентификаторов. Его ограничения заключаются в относительной простоте — он предназначен для генерации чисел, а не для сложной адаптации поведения сложных моделей. Тем не менее, он служит прекрасным теоретическим и практическим примером того, как можно достичь динамической персонализации в полностью состоятельной и автономной системе.

2. Алгоритмическая Адаптация через Приписки: Средний Уровень Гибкости и Контроля. Этот подход, реализованный в методах, таких как Suffix Optimization (SOP), представляет собой более сложную и мощную парадигму, ориентированную на адаптацию больших языковых моделей (LLM) ². Вместо числового семени используется короткая текстовая последовательность (приписка), которая была найдена в результате сложной алгоритмической оптимизации ³. Эта приписка действует как "магический ключ", изменяющий поведение LLM, не затрагивая ее внутренние параметры ³. Персонализация достигается присвоением каждой группе клиентов или каждому клиенту своей уникальной приписки. "Немедленная адаптация" здесь означает, что после дорогостоящей офлайн-оптимизации (которая может занимать минуты на GPU ²) сама процедура применения адаптации сводится к быстрой операции конкатенации строки. Этот метод позволяет реализовывать сложные, многоуровневые политики адаптации, такие как изменение регистра, запрет определенных тем или активация специальных режимов работы. Он представляет собой компромисс: вычислительные затраты сосредоточены в этапе подготовки, а этап выполнения остается чрезвычайно быстрым. Это делает его подходящим для систем, требующих тонкой настройки LLM под нужды конкретных клиентов без необходимости в постоянных обращениях к модели.

3. Парадигма "Скомпилированного ИИ": Высший Уровень Производительности и Предсказуемости. "Скомпилированный ИИ" предлагает наиболее радикальное решение, переводя задачу из области "генерации на лету" в область "детерминированного исполнения" ¹⁰. В этой парадигме LLM используется один раз для генерации статического, оптимизированного и безопасного кода, который затем выполняется без каких-либо дальнейших вызовов к LLM ¹⁰. Персонализация достигается на этапе "компиляции": для каждого клиента или задачи генерируется свой собственный, уникальный блок кода. Это обеспечивает абсолютную предсказуемость, практически нулевую вариативность вывода и экстремально низкую латентность (в миллисекундах ¹⁰). Этот подход является экстремальным примером "динамической генерации", которая происходит вне времени выполнения. Он идеально подходит для критически важных, высоконагруженных задач, где скорость, надежность и экономия ресурсов имеют первостепенное значение, а бизнес-логика четко определена и не требует runtime-гибкости. Компромисс заключается в высокой сложности и затратах на этапе разработки и проверки, а также в потере гибкости во время выполнения.

В таблице ниже представлено сравнение трех основных подходов по ключевым параметрам.

Параметр	Генератор псевдослучайных чисел (ГПСЧ)	Алгоритмическая адаптация (суффиксы)	Скомпилированный ИИ
Основной объект	Числовая последовательность	Поведение большой языковой модели (LLM)	Полный рабочий процесс (код)
Механизм персонализации	Уникальное семя для каждого клиента 17	Уникальная оптимизированная приписка для каждого клиента 2	Генерация уникального детерминированного кода для каждого клиента/задачи 10
Скорость адаптации	Мгновенная (математическая операция)	Мгновенная (конкатенация строки)	Мгновенная (переключение на другой скомпилированный процесс)
Вычислительные затраты	Очень низкие (во время выполнения)	Низкие (во время выполнения); высокие (однократная офлайн-оптимизация) 2	Очень высокие (однократная сложная генерация + строгая проверка) 10
Гибкость	Низкая (ограничена алгоритмом ГПСЧ)	Средняя (зависит от длины и сложности суффикса)	Высокая (может генерировать сложные рабочие процессы)
Предсказуемость	Высокая (детерминированная)	Средняя (детерминированная для заданного суффикса)	Очень высокая (полностью детерминированная)
Архитектурный уровень	Алгоритмический	Прикладной (модификация ввода)	Архитектурный (генерация исполняемого кода)

В заключение, выбор конкретного подхода зависит от специфики задачи, требуемого уровня сложности, доступных вычислительных ресурсов и бизнес-целей. Для простой генерации уникальных числовых последовательностей ГПСЧ является оптимальным решением. Для тонкой и быстрой адаптации поведения LLM под нужды клиентов, не требующей полной перестройки, методы на основе приписок предоставляют мощный и эффективный инструмент. Для задач, требующих максимальной производительности, надежности и предсказуемости, парадигма "Скомпилированного ИИ" открывает путь к созданию высокопроизводительных, детерминированных и экономически эффективных систем. Все эти методы демонстрируют, что в замкнутой системе динамическое, персонализированное и адаптивное поведение является достижимой целью при помощи правильно подобранных алгоритмических и архитектурных решений.

Справка

1. Synthetic Data Generation for Agentic AI | Use Case <https://www.nvidia.com/en-us/use-cases/synthetic-data-generation-for-agentic-ai/>
2. Adaptive Content Restriction for Large Language Models via Suffix ... <https://arxiv.org/html/2508.01198v1>
3. Adversarially Optimised Suffixes <https://www.emergentmind.com/topics/adversarially-optimised-suffixes>
4. SuffixDecoding: Extreme Speculative Decoding for Emerging AI ... <https://suffix-decoding.github.io/>
5. How agentified AI works with large language models <https://www.facebook.com/groups/698593531630485/posts/1252659802890519/>
6. Before the Tool Call: Deterministic Pre-Action Authorization ... <https://arxiv.org/html/2603.20953v1>
7. What makes a method deterministic? What is Input exactly? https://www.reddit.com/r/AskProgrammers/comments/1otp9tx/what_makes_a_method_deterministic_what_is_input/
8. Stateless and Stateful Systems in System Design <https://www.geeksforgeeks.org/system-design/stateless-and-stateful-systems-in-system-design/>
9. Stateful vs. stateless architecture for scalable systems explained <https://aerospike.com/blog/stateful-vs-stateless-architecture-guide/>
10. Compiled AI: Deterministic Code Generation for LLM- ... <https://arxiv.org/html/2604.05150v1>
11. Stateless and deterministic number generation <https://crypto.stackexchange.com/questions/56260/stateless-and-deterministic-number-generation>
12. Do stateless random number generators exist? <https://stackoverflow.com/questions/203382/do-stateless-random-number-generators-exist>
13. Stateless vs. Stateful Architecture: A Comprehensive Comparison <https://www.automq.com/blog/stateless-vs-stateful-architecture-a-comprehensive-comparison>
14. Why Stateful Architecture Is Essential for Agentic AI - ZBrain <https://zbrain.ai/stateful-architecture-for-agentic-ai-systems/>
15. Stateless vs. Stateful Systems in Software Architecture <https://dev.to/abbeymaniak/stateless-vs-stateful-systems-in-software-architecture-nl8>

16. Looking for a replayable / somewhat stateless PRNG algorithm <https://stackoverflow.com/questions/28907136/looking-for-a-replayable-somewhat-stateless-prng-algorithm>
17. avitase/stateless_rand: Stateless and pure functional (pseudo) C++ ... https://github.com/avitase/stateless_rand
18. Pseudo random number generation using a hash function | Devlog <https://maartene.github.io/blog/files/4550a51525c9c2a44d04ac24761e9f71-24.html>
19. Pseudo-Random Number Generator using SHA-256 <https://www.stat.berkeley.edu/~stark/Java/Html/sha256Rand.htm>